

Rime Hackathon Challenge

Build a voice-native product

Build a working product or prototype for a specific user and situation.

Rime-generated speech must be essential to the experience.

A chatbot with a play button is not enough. If removing speech leaves the product mostly intact, voice is not doing enough.

Choose one hard voice problem, define its acceptance test before the demo, and prove that the product handles it under real conditions. Use any speech recognition, language model, orchestration, transport, tool, or interface stack. Rime must provide the primary spoken output.

How judging works

Problem and necessity of voice: 25%

A clear user and problem. Removing speech would make the product materially worse.

Hard voice engineering: 25%

A meaningful voice-specific challenge solved under realistic conditions, such as latency, interruption, pronunciation, multilingual routing, telephony, delivery consistency, or evaluation.

Rime integration and voice experience: 20%

Rime is central to the working product. The selected model, voice, language, [endpoint](#), audio format, and transport fit the situation, and the output is clear and appropriate.

Evidence and reproducibility: 20%

The main claims are backed by a transparent method, committed artifacts, or a repeatable test that measures user-visible behavior.

Demo clarity: 10%

The demo makes the user, problem, working product, Rime's role, stress case, and result easy to understand.

What to submit

Submit one package with a demo and a repository.

- **Demo.** Submit a recorded demo of no more than 4-5 minutes. Show the target user and problem, the normal end-to-end flow, the selected hard voice problem, one deliberate stress or failure case, the result or measurement, and which speech provider is active.
- **Working code.** Submit a source repository that judges can inspect and a working demo link or recording. Edited demos are acceptable, but the demonstrated behavior must exist in the repository. Judges may ask teams to reproduce it.
- **README.** Include setup instructions, architecture, third-party services, known limitations, failure behavior, and the exact [Rime model ID](#), [speaker](#), [language](#), endpoint, audio format, and transport used.
- **Evidence.** Add a short **RIME_EVIDENCE.md** containing the hard voice claim, acceptance test, procedure, result, and limitations. Include a repeatable command, script, or fixture when practical.
- **Configuration hygiene.** Include an environment example with placeholders only, and pass the organizer-provided Rime configuration and secret preflight check.

Choose the voice problem to prove

Start with the voice failure mode, then choose a product where solving it matters. These are directions, not separate scoring tracks. A focused product with one convincingly solved voice problem is stronger than a broad assistant with many shallow features. You may choose one of the paths below or another difficult voice problem with equally clear user impact and an acceptance test.

Before choosing a direction, review the [Voice AI with Rime project catalog](#) to see what builders have already shipped. Use it for inspiration, implementation guidance, and product direction, but do not submit a close reproduction. Extend an idea, combine approaches, improve the voice experience, or solve a harder problem.

- **Perceived response time.** Reduce the delay from the end of the user's turn to the first audible response. Measure the entire user path, including speech recognition, reasoning, buffering, network, synthesis, and playback where they apply. This matters in phone workflows, hands-busy tasks, mobile apps, browser overlays, and other fast exchanges. Good fits also include mobile

voice apps for hands-free work, accessibility, field operations, personal productivity, or other screen-light interaction, and browser extensions or voice overlays that add a persistent voice layer to an existing website, browser workflow, or internal tool without requiring that product itself to add voice.

● **Interruption and recovery.** Stop queued TTS input and local playback promptly, then cancel or fence obsolete model and tool results so they cannot re-enter the conversation. Keep application state consistent with what the user actually heard. This fits full-duplex agents, coaching, simulations, games, and other interactive experiences.

● **Conversation continuity during tool work.** Keep the voice session responsive during lookups, actions, and long-running tasks. Let the user add constraints, request status, interrupt, or cancel without losing context. Prevent delayed results and unheard speech from being applied to the wrong conversational state. This is useful in commerce, support, booking, public services, and healthcare operations.

● **Pronunciation and controlled delivery.** Test names, numbers, codes, addresses, identifiers, and domain vocabulary early, using representative fixtures and before-and-after evidence. Use Brooke Larson's [Writing for the ear guide](#) to match the prompt to the selected voice, keep sentences short, and test punctuation, fillers, repeated words, and false starts by rendering alternatives and listening. A language-learning tutor could repeat or [slow a full response or selected words and phrases](#) where supported; test intelligibility and naturalness at each speed.

● **Multilingual and code-switched speech.** Select compatible models, voices, and language settings deliberately, and test with real target-language material. Relevant products include cross-language support, public services, commerce, and healthcare experiences.

● **Telephony and adverse audio conditions.** Test the actual phone transport, audio format, noise, and device constraints used in the demo. A telephony bridge could serve businesses that still handle booking, ordering, or support by phone; field and limited-device workflows are also strong fits. Browser or studio-quality microphone results do not prove telephone performance.

● **Expressive and persistent voice identity.** Use pacing, wording, voice choice, and supported controls to create delivery that fits the character or situation and remains consistent across turns. This can matter in training, coaching, role-play, games, storytelling, and media.

- **Evaluation and observability.** Build tools that help voice developers reproduce failures, compare implementations, measure regressions, experiment with prompts, or inspect what users actually heard. If the project is a benchmark, follow the rules below.

The interface can be a web app, browser extension, mobile app, phone system, voice overlay, or another format. [LiveKit Agents](#) is the recommended starting point for realtime transport, turn handling, and orchestration across browser, mobile, and telephony applications. Use the [official Rime integration](#) to stream Rime as the primary spoken output. [Qwen Audio Agent](#) remains an optional reference for projects that specifically need a full-duplex voice frontend for long- running coding or task agents.

How to prove the claim

Define the acceptance test before the demo. Run a normal interaction and one deliberate stress or failure case. Measure what the user experiences, not a convenient proxy, and disclose limitations and unsupported input. For prompting or delivery claims, hold the model and voice constant, render at least two text variants, save the clips, and explain which wording or punctuation changed the result.

Full-duplex test example

Introduce a fixed delay into a tool call. While the agent is speaking or waiting, interrupt it and change one part of the request. Verify that queued Rime audio stops promptly, the updated instruction reaches the application, stale tool results are not spoken as current, background work is cancelled or reconciled correctly, and the final spoken response reflects what the user actually heard and requested.

Treat full duplex as a property of the complete application, not the TTS model alone. The application must continue accepting user audio while Rime speech is playing and while tools run.

Benchmark project rules

Community loves comparisons, sometimes even more than customers do. Compare Rime with at least two alternative TTS systems for a clearly defined use case. Separate listening quality, text fidelity, latency, reliability, and controllability. Use comparable, provider-recommended configurations; document voice selection; blind provider identities for listening tests; and distinguish model latency from network latency and warm runs from cold runs. Publish the corpus, exact configurations, generated clips, item-level results, analysis code, and limitations. Report trade-offs instead of one unexplained score, and label small-sample findings as exploratory. Judges will score the fairness and usefulness of the evaluation, not whether Rime wins.

Rime integration and build rules

- **Use a current production configuration.** Choose a compatible model, voice, and language from [Rime's live catalog](#), and test the exact combination used in the demo. Use the current catalog at submission time rather than copying a stale speaker list into the application.
- **Test the shipped path.** Verify the exact endpoint, region, framework, model, audio format, and transport used in the final demo.
- **Protect credentials.** Keep Rime and other service credentials in server-side secrets or an approved provider integration. Never commit credentials to source, documentation, screenshots, recordings, or client code.
- **Make fallbacks visible.** Fallback behavior is allowed and encouraged for resilience, but it must be disclosed. Make the active speech provider observable, and use Rime as the default path in the judged flow.
- **Own the rest of the application.** Rime provides text-to-speech. Your application remains responsible for user input, speech recognition, reasoning, orchestration, state, transport, tools, safety, and evaluation.
- **Design for real use.** Keep spoken turns concise, define clear behavior when dependencies fail or input is unsupported, and use synthetic or de-identified data for healthcare, finance, identity, safety, and other sensitive workflows.

Eligibility and integrity

A submission is not eligible for judging if it:

- Contains no verifiable Rime integration in the submitted code.
- Uses Rime only for a welcome message, final confirmation, optional playback, or other incidental speech.
- Provides only static screens, a concept deck, or a scripted mock without a working product path.
- Omits the required demo.
- Exposes a live credential or other secret.
- Uses a model, voice, or language combination that fails the event preflight and is not corrected before the deadline.

Unverified performance numbers receive no credit. Label cached and uncached measurements separately. Judges will score the shipped code and demonstrated behavior, not unsupported README claims.

Starter resources

- [Voice AI with Rime project catalog](#)
- [Writing for the ear: Prompting your TTS to sound human](#)
- [TTS in five minutes](#)
- [Models](#)
- [Voices and supported languages](#)
- [Live model, voice, and language catalog](#)
- [Regional endpoints](#)
- [Rime prompting guide and drop-in system prompt](#)
- [LiveKit with Rime](#)
- [Qwen Audio Agent \(optional realtime runtime\)](#)
- [Playback speed controls](#)